

Docker Stress Test Lab Report

1. Objective

To build a Docker container for a Python-based stress testing application and apply CPU and memory limits using both CLI and Docker Compose methods. The goal is to understand container resource control and observe system behavior under constrained environments.

2. Problem Statement

- Run a Python application inside a Docker container
- Ensure the application executes successfully
- Apply resource limits:
 - CPU limited to 1 core
 - Memory limited (e.g., 512MB or 3GB depending on configuration)
- Validate that limits are enforced
- Troubleshoot issues encountered during setup and execution

3. Environment Setup

- Operating System: Windows
- Docker Desktop installed
- Python-based application (`stress.py`)
- Docker CLI and Docker Compose used

4. Application Overview

The Python script performs:

- CPU stress using multiprocessing

- Memory stress using NumPy
- Optional GPU detection using OpenCL

It accepts arguments:

- `--cpu` → number of CPU threads
- `--mem` → memory in GB
- `--time` → duration in seconds

5. Steps Performed

Step 1: Project Setup

Created a project folder:

`docker-stress-app/`

Files inside:

- `Dockerfile`
- `requirements.txt`
- `stress.py`

Step 2: Dockerfile Creation

`FROM python:3.11`

`WORKDIR /app`

`COPY . .`

`RUN pip install --no-cache-dir -r requirements.txt`

`CMD ["python", "stress.py"]`

Step 3: Requirements File

```
numpy  
psutil
```

Step 4: Build Docker Image

Command:

```
docker build -t stress-app .
```

Step 5: Run Container

```
docker run stress-app
```

Step 6: Run with Resource Limits (CLI Method)

```
docker run --cpus="1" --memory="512m" stress-app
```

OR:

```
docker run --cpus="1" --memory="3g" stress-app
```

Step 7: Check Running Containers

```
docker ps
```

Step 8: Check All Containers

```
docker ps -a
```

Step 9: Stop Container

```
docker stop <container_id>
```

Example:

```
docker stop 8407c3d52081
```

Step 10: Force Stop (if needed)

```
docker kill <container_id>
```

6. Docker Compose Method (Configuration-Based Limits)

docker-compose.yml

```
version: "3.9"
```

```
services:
```

```
  app:
```

```
    build: .
```

```
    container_name: stress-app-compose
```

```
    mem_limit: 512m
```

```
    cpus: 1.0
```

Run with Compose

```
docker-compose up --build
```

Stop:

CTRL + C

7. Issues Encountered

Issue 1: Invalid requirement error

Error:

Invalid requirement: 'optional pyopenc1'

Cause:

- Incorrect entry in requirements.txt

Resolution:

- Removed invalid line
- Kept only:

numpy
psutil

Issue 2: Missing stress.py file

Error:

python: can't open file '/app/stress.py'

Cause:

- File not present in project directory

Resolution:

- Created stress.py and placed it inside project folder
- Rebuilt Docker image

Issue 3: Path navigation confusion

Cause:

- Incorrect directory usage in CLI

Resolution:

- Used correct command:

cd docker-stress-app

8. Information Gathering & Observations

- Docker Desktop used to monitor:
 - CPU usage
 - Memory usage
- `docker ps` used to track running containers
- Resource limits verified through:
 - Docker Desktop UI metrics
 - Container behavior under load

9. Results

- Container successfully built and executed
- CPU and memory limits applied via CLI and Docker Compose
- Application executed within constrained resources
- Stress test demonstrated controlled resource usage

Common Command:

Build image

```
docker build -t stress-app .
```

Run container

```
docker run stress-app
```

Run with limits

```
docker run --cpus="1" --memory="512m" stress-app
```

List containers

```
docker ps  
docker ps -a
```

Stop container

```
docker stop <id>
```

Compose run

```
docker-compose up --build
```